# Questions and answers

Every question the committee and the supervisor asked, plus the follow-ups to expect at the defence – each with a short answer and where the proof is

**How to read this.** One question per block, in plain words, with the measured number that answers it. "Proof" names the section of `docs/Architecture_and_Benchmark_Report.pdf` (here: Benchmark report) or the output folder the number comes from. Questions about the thesis text – literature review, chapter order – are answered with what is ready for the author; the text itself is the author's.

## 1. The main question

### Q1. If an AI model can look at an invoice image and extract the information directly, why should we still use OCR first and then send the extracted text to an LLM?

Committee, 50 % review: "justify scientifically why a dedicated OCR→LLM pipeline is still necessary or advantageous (hardware, cost, control)"

**Short answer: on the hardware this thesis targets, the OCR → LLM pipeline is about 20 times faster and cheaper per invoice than a vision-language model, which is somewhat more accurate. And the two can be combined: the pipeline handles every invoice, the vision model only the few the pipeline is unsure about – that reaches the vision model's accuracy at a fraction of its cost.**

**We measured it** – same 600 invoices, same RTX 3050 (4 GB), same scorer. The vision-language model (VLM) is Qwen2-VL-2B, which reads the image directly.

| System | Reads | Time per invoice | Invoices per minute | GPU memory | F1 all | F1 real invoices |
|---|---|---|---|---|---|---|
| A – OCR → LLM, one at a time | OCR text | 1.13 s | 53 | 1067 MB | 0.941 | 0.892 |
| B – OCR → LLM, optimised | OCR text | 0.20 s | 302 | 1996 MB | 0.941 | 0.894 |
| VLM – Qwen2-VL-2B, 4-bit | image | 3.90 s | 15 | 1745 MB | 0.966 | 0.930 |

A and the VLM: one invoice at a time. B: total time for 600 invoices divided by 600, because B works on several invoices at once. A and B: mean of 3 runs; VLM: one run.

**Four reasons the OCR → LLM pipeline still makes sense**

1. **Speed and cost.** B needs 0.20 s per invoice, the VLM 3.90 s – 20 times more. That is 302 against 15 invoices per minute, and about €0.0019 against €0.038 per 100 invoices at the same GPU price (VLM cost: own arithmetic from the measured time).
2. **It fits the hardware, with room to work in parallel.** The VLM only fits the 4 GB card in 4-bit and with the image reduced to at most 1024 visual tokens – its normal 16-bit weights alone take 4.4 GB. More resolution costs more: 2048 tokens needed 2.3 GB and 5.2 s per invoice. OCR runs on the CPU and leaves the GPU to the language model, so B can read the next invoices while the GPU answers the current ones – that overlap is where most of B's speed comes from. A VLM does all its work on the GPU, so there is nothing to overlap.
3. **Control.** OCR produces text that can be checked: the total is verified arithmetically (subtotal + tax = total), invoices with a missing or unreadable field are flagged for a person, and every value can be traced to a line of the invoice. A VLM gives an answer with nothing in between to check it against.
4. **The VLM's accuracy can be borrowed where it matters.** B's review flag sends 45 of the 600 invoices (7.5 %) to a person. On those 45, B is fully correct 3 times and the VLM 39 times. Sending only those to the VLM gives 541 of 600 fully correct invoices – the same as the VLM on every invoice – for about one thirteenth of the VLM's GPU time (estimated from the stored results).

**What we do not claim.** The VLM is more accurate (real invoices: invoice number 0.99 against 0.94). Only one small VLM with one prompt was measured; larger or cloud VLMs were not. The literature has evidence on both sides – see `Literature_Sources.pdf`, section 2.

Proof: Benchmark report, section 7; outputs/2026-10-20_23-54-32_vlm_baseline; outputs/2026-10-21_00-35-48_vlm_resolution_probe_20docs.

# 2. The committee's 50 % review

### Q2. Is the topic still relevant now that vision-language models exist?

Committee, 50 % review

Yes. Q1 shows the measured trade-off: on a 4 GB GPU the OCR → LLM pipeline is 20 times faster and cheaper than a VLM, and the two combine well. Making such pipelines fast on modest hardware is exactly the thesis question.

Proof: Benchmark report, sections 6.1 and 7.

### Q3. Which bugs led to the incorrect results you presented?

Committee, 50 % review

Twelve defects were found and fixed. The four that changed the results most:

- The model was asked to answer in JSON, but Flan-T5 cannot write { or }. Every answer silently came from a regular expression, not from the model.
- The accuracy script compared the wrong object with the ground truth, so every field looked empty.
- Amounts were compared on their first digits only: $6,270 matched any amount from $6,000 to $6,999.
- A and B were scored by different scripts with different thresholds (0.7 against 0.9), so they could not be compared.
- Also: the "first 50" invoices were picked by text sorting (1, 10, 100 …); A ran on the CPU, B on the GPU; the "4-bit" script loaded 8-bit; throughput was calculated wrongly; and a date parser read "April 19,2024" with the current year.

Proof: docs/CHANGES.pdf; Architecture_Report.pdf, section 3.1.

### Q4. Were the bugs fixed and everything re-run on the full dataset?

Committee, 50 % review

Yes. All 600 invoices, 12 configurations, 3 runs each, with one shared scorer for everything. 33 runs that other programs disturbed (detected by OCR time per invoice) were repeated; they are kept for transparency. 66 automated tests check the scoring and parsing code.

Proof: outputs/2026-10-18_23-52-41_full_benchmark (summary.md, discarded/); Benchmark report, section 5.3.

### Q5. The improvements were called trivial. What does the work show now?

Committee, 50 % review

Measured on all 600 invoices:

- B processes 302 invoices per minute against 53 for A – 5.7 times more – at the same accuracy (F1 0.941), and costs 5.8 times less.
- Accuracy rose from F1 0.85 to 0.94 through two changes found by error analysis: reading the OCR text row by row, and checking the total arithmetically. Correct totals on real invoices went from 34 % to 83 % (B).
- A review flag accepts 92.5 % of invoices automatically, and 90.5 % of those are fully correct.
- Routing thresholds were derived from the data, not guessed, and validated on fresh runs.
- A vision-language model and a second OCR engine (EasyOCR) were measured on the same hardware, and latency was measured with invoices arriving one at a time.

Proof: Benchmark report, sections 6–8.

### Q6. Where is the analytical part – the literature review of other authors' work and existing solutions?

Committee, 50 % review

The author writes it as chapter 1. Ready for it: Literature_Sources.pdf with 150 checked sources in 8 topics (VLM versus OCR, OCR tools, caching, compression, asynchronous processing and batching, evaluation metrics, related theses), and Thesis_Outline.pdf, chapter 1, which lists the questions the chapter must answer.

Proof: docs/Literature_Sources.pdf; docs/Thesis_Outline.pdf.

### Q7. The structure was illogical – the dataset came after the results.

Committee, 50 % review

The new order is analysis → system design → dataset and evaluation method → results → conclusions. Thesis_Outline.pdf follows that order chapter by chapter; the author writes the chapters.

Proof: docs/Thesis_Outline.pdf.

## 3. The supervisor's technical questions

### Q8. Is Architecture A a real AI pipeline now, not regular expressions?

Supervisor, before the 50 % submission

Yes. A runs Tesseract OCR, then a MiniLM sentence-embedding model that keeps the relevant lines, then Flan-T5, which answers three questions per invoice (number, date, total). The old version's answers came from a regular expression because of the JSON bug (Q3).

Proof: Benchmark report, section 3; pipeline/arch_a.py.

### Q9. What exactly is the difference between A and B?

Defence

Same OCR, same models, same prompts, same checks – only the organisation differs. A does one invoice at a time, start to finish. B looks each invoice up in a cache first, reads three invoices at once on the CPU, and sends them to the GPU in groups of 8 (24 questions in one model call). A router can send simple invoices to a smaller 4-bit copy of the model. That is why the answers are the same and only the speed differs.

Proof: Benchmark report, sections 3–4.

### Q10. OCR and the LLM block the program. How can B be asynchronous?

Supervisor, 25 % feedback

Exactly as suggested: an asyncio event loop hands OCR to a pool of three worker processes and model work to one GPU thread, both through loop.run_in_executor. The loop itself only schedules, so OCR of the next invoices and the GPU's work on the current batch overlap. Measured value of each part: parallel OCR 2.8 times faster, GPU batching 2.1 times faster.

Proof: Benchmark report, sections 4.3 and 6.4.

### Q11. Why SQLite and not Redis, as planned? Why not Celery or Kafka?

Supervisor, 20 % feedback

The system runs on one computer. SQLite needs no server, keeps its data after a restart, and a lookup takes about 1 ms – the same 600 invoices come back from the cache in 0.59 s instead of 119 s. Redis is the better choice when several machines share a cache; Celery or Kafka add a message broker that brings no speed on one machine. This is a deliberate choice, not an omission.

Proof: Benchmark report, sections 4.1 and 6.10.

### Q12. How is the model compressed, and with which tool?

Supervisor, 25 % and 75 % feedback

With bitsandbytes, loaded through Hugging Face transformers: 4-bit NF4 with double quantization, and 8-bit LLM.int8. Half precision (fp16) was measured as well. GGUF / llama.cpp, the other tool named, was not needed because bitsandbytes runs inside the same PyTorch pipeline on the GPU.

Proof: Benchmark report, section 2.4; pipeline/llm.py.

### Q13. Does 4-bit quantization reduce latency and memory on the RTX 3050?

Supervisor, 75 % feedback

It reduces memory, not time. Every invoice on one model:

- fp32: 126.1 s for 600 invoices, 1645 MB, F1 0.941
- 4-bit: 125.8 s, 1102 MB (one third less), F1 0.941
- fp16: 123.5 s, 1049 MB, F1 0.941 – the best option on this GPU
- 8-bit: 167.8 s, 1176 MB, F1 0.939 – the slowest
- The model has 248 million parameters; at that size unpacking 4-bit weights costs as much time as the smaller weights save.

Proof: Benchmark report, section 6.8.

### Q14. How does the routing decide, and how were the thresholds chosen?

Supervisor, 25 % and 75 % feedback

Rule: if the OCR text has at most T tokens and the mean OCR confidence is at least C, the invoice goes to the 4-bit model; otherwise to the full model. Start values T = 300, C = 85 sent 505 invoices to 4-bit and 95 to the full model.

The final values come from the 600 results: every invoice was run through both models, and a grid search picked the setting that sends the most invoices to 4-bit while losing less than 0.5 percentage points of accuracy. Result: T = 525, C = 70, so 99.8 % go to 4-bit – the two models disagree on only 10 invoices. Three fresh runs confirmed it (F1 0.941).

Proof: Benchmark report, section 6.9; outputs/2026-10-18_23-52-41_full_benchmark/routing_analysis.md.

### Q15. How is accuracy measured?

Supervisor, 25 % and 75 % feedback

F1 per field (invoice number, date, total), from true positives, false positives and false negatives. Invoice numbers count as equal at ≥ 90 % similarity (RapidFuzz – the fuzzy matching asked for), dates are compared as calendar dates, amounts as numbers. Real and synthetic invoices are reported separately.

Proof: Benchmark report, section 5.4; evaluation/scorer.py.

### Q16. Where does the ground truth come from? Did you use a teacher model such as GPT-4?

Supervisor, 20 % and 75 % feedback

Synthetic invoices: the generator writes the correct answers together with each image. Real invoices: labelled by hand. A check found 26 real labels holding the amount before tax instead of the amount due; they were corrected only where net + tax = gross on the invoice, and the originals are kept. A teacher model was not needed because labels already existed, and the arithmetic check is exact and auditable.

Proof: Benchmark report, section 5.1; ground_truth_backup_20261018/.

### Q17. How did you reach about 600 documents?

Supervisor, plan for the 50 % submission

300 synthetic invoices from a generator script and 300 real invoices (public templates and scans), 600 in total.

Proof: Benchmark report, section 5.1.

### Q18. Tesseract or EasyOCR – which is better?

Supervisor, plan for the 50 % submission

Both read all 600 invoices into the same pipeline. Tesseract: F1 0.941, 0.89 s per real invoice on the CPU, no GPU memory. EasyOCR: F1 0.934, 8.0 s per real invoice on the GPU, 3.9 GB of GPU memory. Tesseract is faster, slightly more accurate here, and leaves the GPU free.

Proof: Benchmark report, section 6.12; outputs/2026-10-20_01-52-39_ocr_comparison.

### Q19. Which hardware, and can the results be reproduced?

Supervisor, 20 % feedback

Laptop with an NVIDIA RTX 3050 (4 GB), 12 CPU threads, 16 GB RAM, Windows 11. Every run is repeated three times; each run folder records the exact command, settings, hardware and library versions; models load offline; one command re-runs everything. Disturbed runs are detected and repeated (Q4).

Proof: Benchmark report, sections 5.3 and 5.5; README.md.

### Q20. What does processing cost?

Supervisor, e-mails

At €0.35 per GPU-hour: A €0.0110 per 100 invoices, B €0.0019 – 5.8 times less. Re-processing cached invoices costs practically nothing.

Proof: Benchmark report, section 6.1.

## 4. Follow-up questions to expect at the defence

### Q21. Is B also faster for a single invoice?

Defence

Not by itself – B is built for many invoices. For one invoice both take about the same compute. Measured with invoices arriving one at a time:

At 30 per minute, B as benchmarked waits until 8 invoices are read, so the median wait is 7.9 s against 1.7 s for A. With dynamic batching (the GPU takes whatever is ready, up to 8) B's median drops to 0.95 s. At 120 per minute A can no longer keep up (median 22.8 s), B needs 3.0 s and B with dynamic batching 1.2 s.

Proof: Benchmark report, section 6.11; outputs/2026-10-20_01-01-33_latency_60docs.

### Q22. Why does B use more memory?

Defence

B keeps two copies of the model on the GPU (4-bit and full) and processes 8 invoices at once. Peak GPU memory 2.0 GB against 1.07 GB for A; RAM 2.0 GB against 1.4 GB. Both stay well inside the 4 GB GPU and 16 GB RAM.

Proof: Benchmark report, section 6.5.

### Q23. Why Flan-T5-base and not a bigger model?

Defence

It fits the 4 GB GPU twice – full precision and 4-bit – so the comparison measures the architecture, not a lack of memory. A bigger model would likely be more accurate; that is future work. A and B share the model, so the comparison is fair either way.

Proof: Benchmark report, section 2.2.

### Q24. Why are real invoices less accurate than synthetic ones?

Defence

The synthetic invoices share one clean layout (F1 0.99); real invoices have tables, many formats and scans (F1 0.89). That is why both are always reported separately.

Proof: Benchmark report, section 6.6.

### Q25. What errors remain, and does the review flag catch them?

Defence

Mostly a real value copied from the wrong place on the page – a nearby number taken as the invoice number, or day and month swapped. The review flag catches 42 of the 95 invoices with an error; an estimated label check (the number must appear next to "Invoice" or "No.") would catch 64.

Proof: Benchmark report, sections 6.7 and 8.

### Q26. How do you know the timings are valid?

Defence

OCR time per invoice is about 0.55–0.63 s on an idle machine. Whenever another program interfered – two demo runs started during the benchmark, a background process that used up to 8 of the 12 CPU threads – OCR time rose, and every affected run was repeated. All 33 discarded runs are kept with the reason in the folder name.

Proof: outputs/2026-10-18_23-52-41_full_benchmark/discarded; docs/RESULTS.md, section 8.

### Q27. What would you improve next?

Defence

Ordered by how well the data supports it:

- Dynamic batching in B – measured: median latency at 30 invoices per minute 7.9 s → 1.0 s.
- Faster OCR or more CPU cores – B is limited by OCR, not by the GPU.
- fp16 instead of 4-bit on the fast route – faster model step, least memory, same accuracy.
- Label checks in the review flag – estimated 64 instead of 42 of 95 errors caught.
- Send only flagged invoices to a VLM – estimated 541 instead of 505 of 600 fully correct.

Proof: Benchmark report, section 8.